Software Design Document

1.Project Information

Project Name: Fantasy Tactics League

Document Type: Software Design Document (Assignment 2)

Authors:

- Muhammet Emir Dizdar (211401028)
- Doruk Dolaşık (211401030)
- Serhan Tezcan (211401024)

Document-Specific Task Matrix

Task matrix:

Task	Responsible Member(s)
System Overview & Context	Serhan Tezcan
Architectural Design & Diagrams	Muhammet Emir Dizdar
Design Patterns Documentation	Doruk Dolaşık
Data & Interface Design	Serhan Tezcan
Final Review & Editing	Muhammet Emir Dizdar

2. Introduction & System Overview

2.1 System Overview

Fantasy Tactics League is a standalone web application designed as a simulation platform for football management. It allows users to create and manage virtual football teams composed of real-life Premier League players. The system emphasizes a modular architecture using a Python-based backend and a React frontend to demonstrate the practical application of various software design patterns. By integrating real-world football data, the platform provides an immersive experience in team building, strategic transfers, and performance tracking.

2.2 System Context

The system operates as a central hub that interacts with external entities to facilitate the fantasy football experience.

- Users (Fantasy Managers): Interact with the system to build squads, manage transfers, and view rankings.
- Football Data API (football-data.org): Provides critical real-time data, including player metadata, match results, and fixtures.
- Local Database (SQLite): Ensures data persistence for user team configurations, player costs, and historical points.

2.3 Key Features and Functionality (Selected Use Case: Create New Fantasy Team)

For this assignment, the system focuses on the "Create New Fantasy Team" use case:

- Player Selection: Users select a squad of 11 players from a list of available Premier League players categorized by position.
- Real-time Budget Tracking: The system dynamically updates and displays the remaining budget as players are added to the squad.
- Constraint Validation: Before saving, the system validates the team against a fixed virtual budget of £100.0m and specific squad size/positional requirements.
- Data Persistence: Once validated, the new team record is saved to the database, and the user is enrolled in the league ranking system.

2.4 Assumptions and Dependencies

- API Availability: The system assumes that the external football-data.org API is accessible and provides player data in a valid JSON format.
- Connectivity: It is assumed that the server has an active internet connection to fetch real-world statistics.
- Rate Limits: To avoid API rate limit violations, the system depends on data caching or pre-defined data-fetching scripts.
- Browser Compatibility: The application is expected to run on modern web browsers across Windows, Linux, and macOS platforms.

3. Architectural Design and Component Design

This section details the holistic structural architecture of the **Fantasy Tactics League** application, outlining the layered separation within the system and the relationships between developed components through technical descriptions and architectural diagrams.

3.1 Architectural Design

The project integrates a **Layered Architecture** combined with **Client-Server (API-Driven)** patterns to ensure complete isolation of client-server interactions and data-fetching scenarios.

3.1.1 System Architecture Diagram (High-Level)

The system is divided into three independent layers and an external integration ring:

1. **Data Fetching & ETL Layer:** Consists of autonomous scripts like fpl_fetcher.py that retrieve raw JSON/CSV data from external APIs and clean it for the system.
2. **Data Access Layer (DAL):** Manages the **SQLite** database (fantasy.db) and schema administration via database_setup.py.
3. **Business Logic Layer (BLL):** Contains the core OOP domain models, the validation engine (team.py, strategies.py), and the **FastAPI** controller (api.py) which acts as the RESTful interface.
4. **Presentation Layer (UI):** A decoupled frontend built with **React.js** and **TailwindCSS**.

   Figure 1: High-Level System Architecture showing the decoupled interaction between the External Data Layer, the SQLite Persistence Layer, and the API-driven Frontend communication

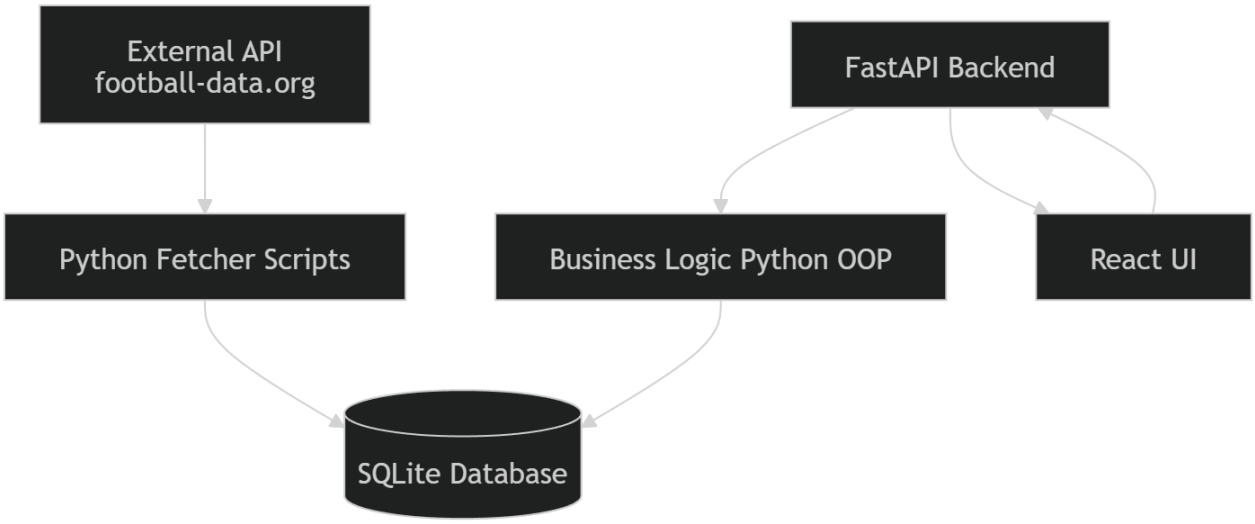

Figure 1

3.1.2 Rationale for Architectural Decisions

- **Decoupled Client-Server:** By not using server-side rendering (SSR), the API remains independent. This allows for isolated testing and ensures that the backend can serve future platforms (e.g., iOS/Android) without modifications.
- **External Data Abstraction (ETL Layer):** The FastAPI server is not directly coupled with the official FPL APIs. It only interacts with the cleaned data in fantasy.db, while the task of parsing and formatting external data is handled by autonomous background scripts.

3.2 Component Design

The Business Logic Layer is divided into specialized sub-modules following the **Single Responsibility Principle** to ensure modularity.

3.2.1 Subsystems and Modules (Responsibilities)

- **models.py (Domain Models):** Responsible for creating data containers with strict type definitions (e.g., Goalkeeper, Defender). These objects store state (stats, cost) without containing complex behaviors.
- **factories.py (Object Creator):** Acts as the instantiation subsystem by converting database row dictionaries into appropriate Model objects.
- **team.py (Central Command / Aggregate Root):** The core object representing the FPL Team. It manages player arrays, observer notifications, and validation strategies. All team-related operations (add, delete, validate) are routed through this component.
- **strategies.py (Rules Engine):** Executes business rules such as budget limits and positional constraints.
- **observers.py (Event Communication):** Handles side-effects like logging or sending asynchronous messages when a team is saved or a rule is violated.
- **singletons.py (Infrastructure Providers):** Ensures memory-safe, single-reference distribution of I/O operations (database and network) across all modules.

3.2.2 Interfaces and Component Dependencies

The communication between components follows a **Unidirectional Data Flow**:

1. **FastAPI (api.py)** receives an HTTP request and passes DTOs to the **factories.py**.
2. The factory produces **Player Objects** which are then added to the core list in **team.py**.

3. **team.py** executes the rules via **strategies.py** and triggers **observers.py** for UI logging or status updates.

   Figure 2: Component Dependency Diagram illustrating the modular structure of the Business Logic Layer, highlighting how core modules interact through defined interfaces to maintain low coupling

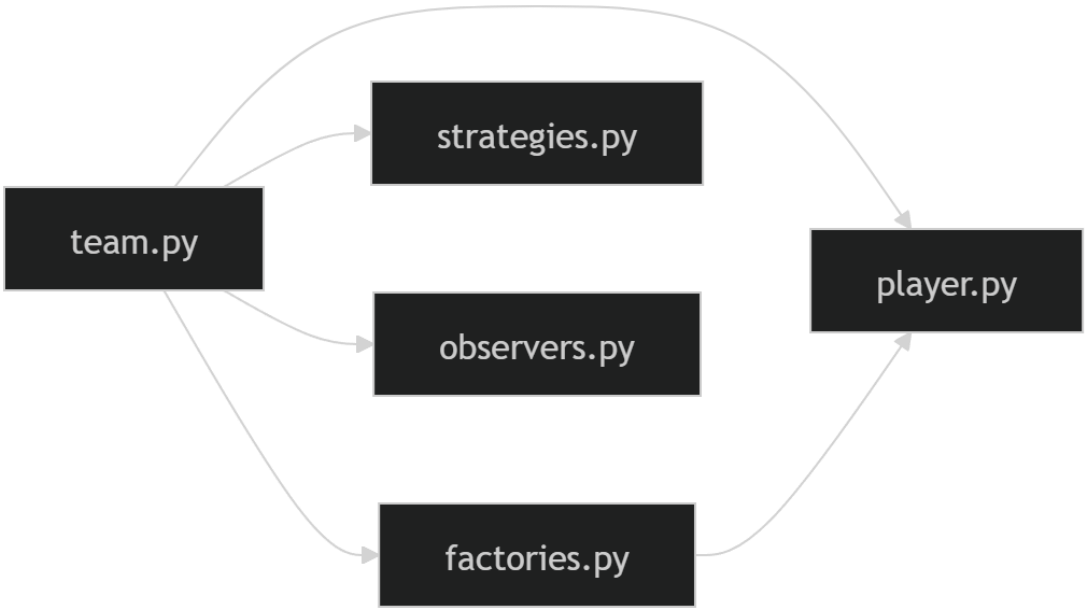

Figure 2

3.2.3 Communication Contracts

To maintain loose coupling, components interact through standardized interfaces:

- **strategies.py** exposes only the validate(team) -> Tuple[bool, str] signature to the rest of the system.
- **team.py** (The Subject) broadcasts events to observers using a standardized notify(event_type: str, **kwargs: Any) method.
- **factories.py** provides a static create_player(data: dict) -> Player method for external use.

4. Data Design and Design Patterns

This section details the backend architecture, the persistent data model, and the technical implementation of the Software Design Patterns integrated into the **Fantasy Tactics League** platform.

4.1 Data Design

The system utilizes **SQLite** as the local relational database to ensure data persistence, efficient querying, and seamless integration with real-world football statistics.

4.1.1 Data Model / ER Diagram Logic

The system is structured around four primary entities:

- **players Table:** Stores the master list of Premier League players, including names, teams, costs, and position identifiers (1:GK, 2:DEF, 3:MID, 4:FWD).
- **stats Table:** Contains weekly statistical performance data for players. It maintains a **1:N (One-to-Many)** relationship with the players table, as one player generates different stats each Gameweek (points, goals, assists, etc.).
- **user_teams Table:** Records the fantasy squads created by users, tracking the team name, total budget used, and creation timestamp.
- **team_players Table:** A **junction table** that resolves the **N:M (Many-to-Many)** relationship between user teams and players. It also includes a flag to designate which player is the team captain.
- **Figure 3:** Entity-Relationship Diagram (ERD) illustrating the database schema, including player metadata, weekly statistics, and the relational mapping between users and their squads.

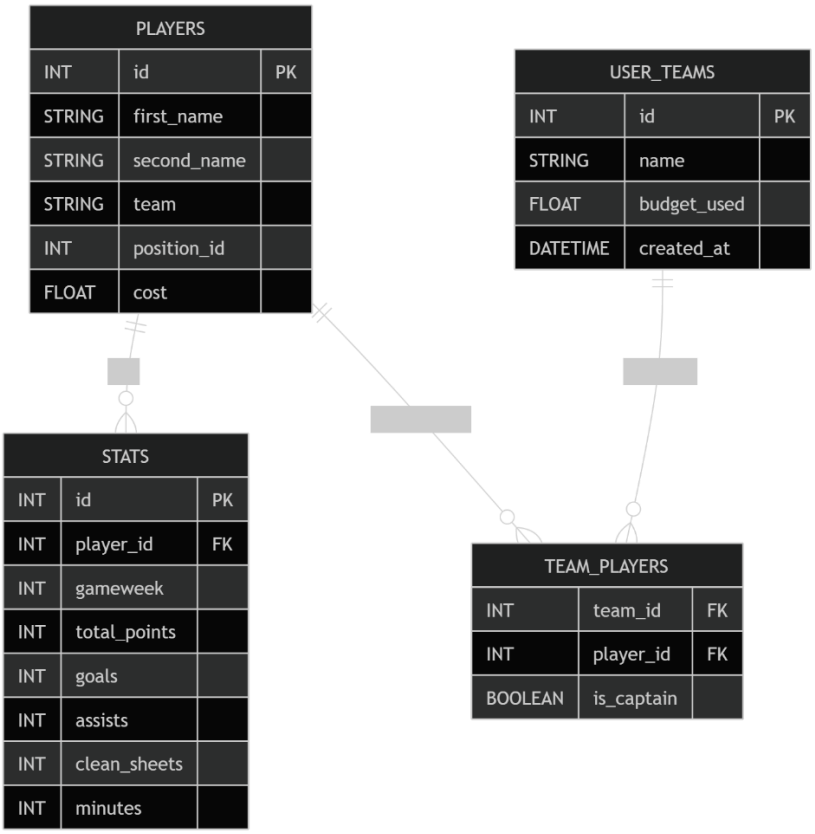

Figure 3

4.1.2 Data Flow Diagram (DFD)

1. **External Data Fetching:** Real-season statistics are retrieved via Python scripts (fpl_fetcher.py) and stored in the players and stats tables.
2. **User Interaction:** The React UI sends selected player IDs to the FastAPI backend via a POST request to the /team endpoint.
3. **OOP Conversion:** Raw data from SQLite is transformed into Object-Oriented models using the **Factory Pattern**. The CurrentTeam object then processes these through the validation engine.
4. **Database Persistence:** Valid squads are saved to the user_teams and team_players tables. If validation fails, the **Observer Pattern** triggers a 400 Bad Request response to the UI with detailed error reasoning.

   Figure 4: Data Flow Diagram (DFD) illustrating the end-to-end data lifecycle, from external API consumption via ETL scripts to user-driven team validation and persistent storage.

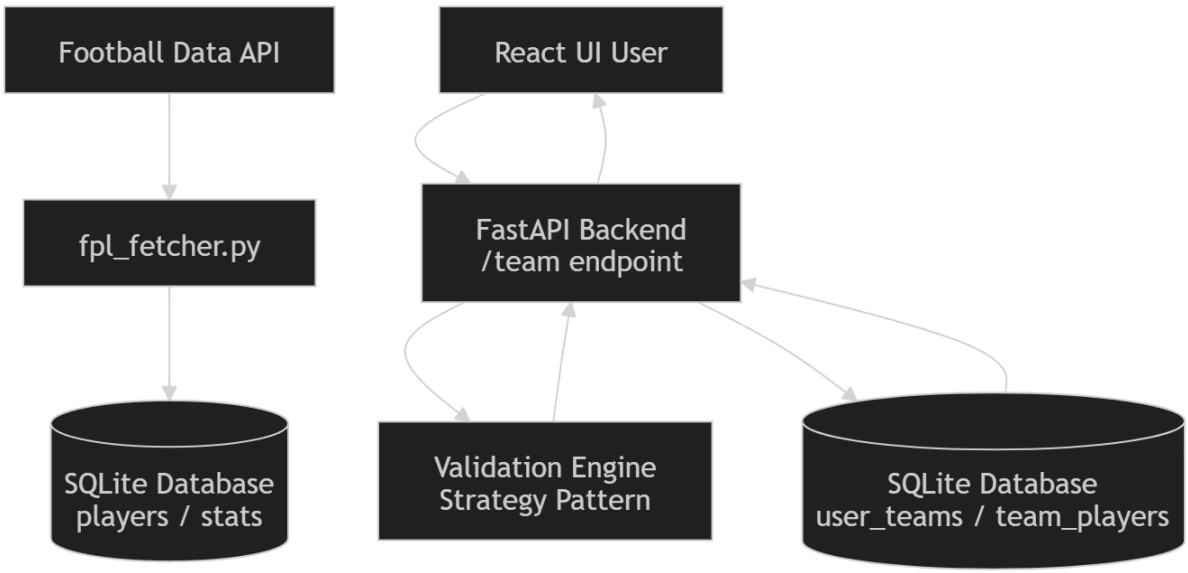

Figure 4

4.1.3 Data Validation Rules

To maintain the integrity of the fantasy format, the following strict rules are implemented via the **Strategy Pattern**:

- **Budget Limit:** Total squad cost cannot exceed **£100.0M**.
- **Squad Size:** Teams must consist of exactly **15 players**.

- **Position Requirements:** Squads must contain exactly 2 Goalkeepers, 5 Defenders, 5 Midfielders, and 3 Forwards.
- **Team Limitation:** A maximum of **3 players** can be selected from the same real-life Premier League club.

4.2 Design Patterns

The platform adheres to "Gang of Four" (GoF) principles to ensure the system is modular, maintainable, and extensible.

4.2.1 Factory Method Pattern

- **Application:** PlayerFactory class and its create_player() method.
- **Justification:** This pattern instantiates specific player subclasses (Goalkeeper, Defender, etc.) based on their position_id. It prevents "Tight Coupling" by moving complex instantiation logic away from the core business logic. It follows the **Open-Closed Principle**, allowing for new positions to be added without modifying existing core code

  Figure 5: Class Diagram of the Factory Method Pattern implementation, showing how specific player positions (Goalkeeper, Defender, etc.) are instantiated through a centralized creator

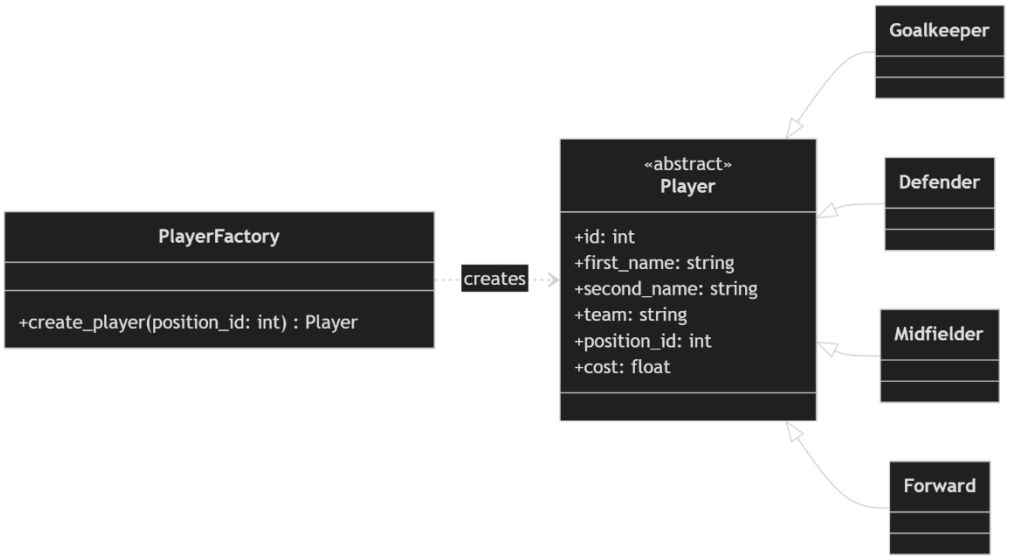

Figure 5

4.2.2 Strategy Pattern

- **Application:** Modular validation algorithms for budget, squad size, and team limits.
- **Justification:** Since fantasy rules can change annually, validation logic is isolated into a family of interchangeable algorithms (ValidationStrategy). This "plug-and-

play" architecture allows rules to be added or modified independently of the CurrentTeam class.

Figure 6: Class Diagram of the Strategy Pattern used for the validation engine, enabling modular and interchangeable rules for budget and squad composition

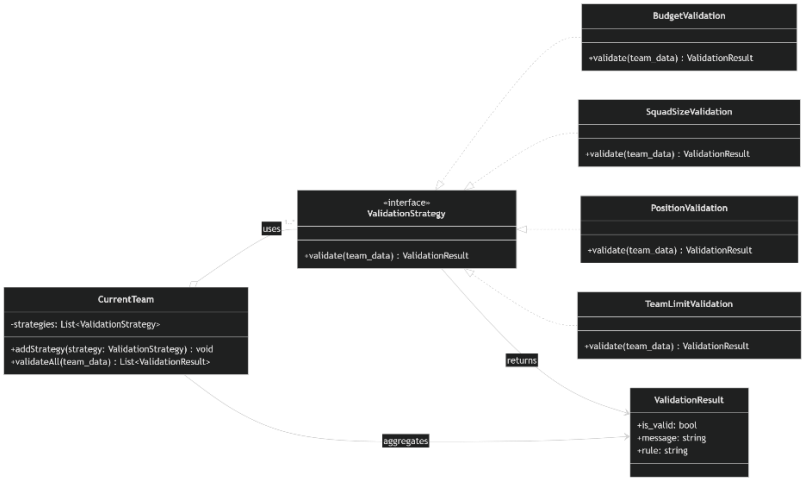

Figure 6

4.2.3 Observer Pattern

- **Application:** RealTimeUIUpdater acting as an Observer to the CurrentTeam (Subject).
- **Justification:** This ensures "Loose Coupling" between the core logic and the user interface. When a player is added or validation fails, the system triggers notifications (e.g., "PLAYER_ADDED", "VALIDATION_FAILED") to all registered observers without the core needing to know the details of the UI implementation.

4.2.4 Singleton Pattern

- **Application:** DatabaseManager and APIService classes.
- **Justification:** To prevent excessive memory usage and socket exhaustion, these classes ensure that only one shared instance of the database connection or API client exists throughout the application lifecycle.

4.3 Implementation Notes

- **Separation of Concerns:** The architecture clearly separates the React Frontend, FastAPI Backend, and Python OOP Business Core through defined API boundaries.
- **Type Hinting:** All abstractions use Python **Type Hints** (e.g., -> Tuple[bool, str]) to ensure strict object communication contracts.

- **Graceful Failure:** The system implements "Fail Gracefully" mechanisms, such as safe_float and dict.setdefault(), to handle potential schema mismatches or missing API data.

5. User Interface Design and External Interfaces

This section outlines the presentation layer's design, the integration with external data sources, and the performance strategies implemented to ensure a smooth user experience for the "Create New Fantasy Team" use case.

5.1 User Interface Design

The frontend is developed as a **Single Page Application (SPA)** using **React** and **TailwindCSS**, prioritizing a responsive and intuitive workspace for fantasy managers.

- **Workspace (Player Picker):** The left panel features a comprehensive player pool with a built-in search engine. To prevent performance lag, the list is categorized by position (ALL, GKP, DEF, MID, FWD) and sorted by cost, allowing users to filter through the data efficiently.
- **Interactive Pitch Layout:** The right panel provides a visual representation of the team, divided into "Starting XI" and "Substitutes". The interface automatically places the first goalkeeper and ten outfield players in the starting lineup, while additional selections are displayed with a "bench" effect.
- **Real-time Feedback:** Validation results from the backend's **Strategy Pattern** are immediately reflected on the UI. Users receive direct feedback via JavaScript toast messages or alerts (e.g., "Too many players selected from Arsenal") to correct errors before submission.

Figure 7: High-fidelity User Interface of the Fantasy Pro platform, showcasing the interactive pitch layout, squad composition, and real-time budget tracking.

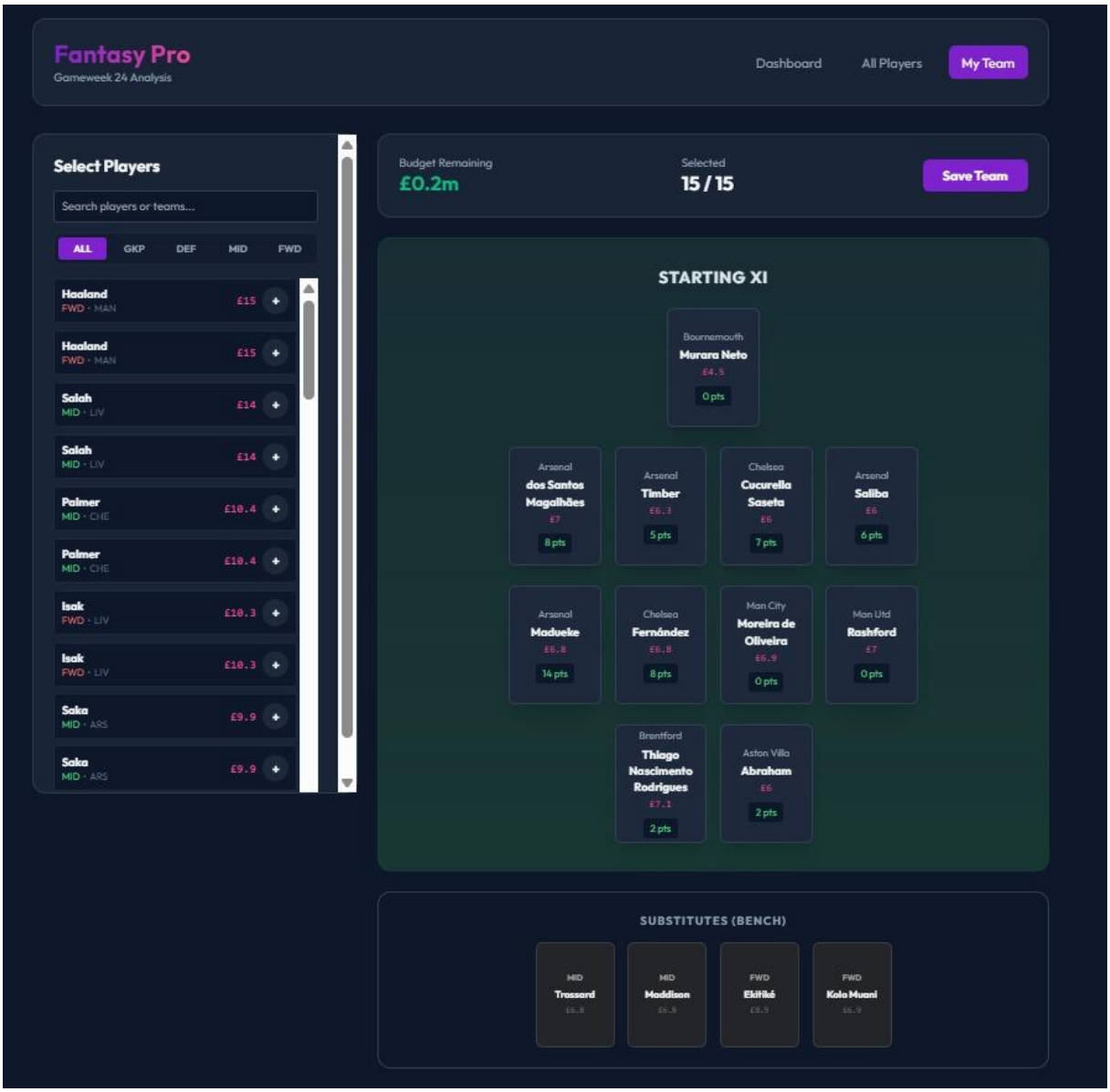

Figure 7

5.2 External Interfaces

The application functions by harmonizing real-world data with internal management logic.

- **Third-party API (Official FPL API):** Rather than using mock data, the system consumes the official Fantasy Premier League API (/api/bootstrap-static/). Python scripts monitor this endpoint to fetch up-to-date player IDs, costs, and weekly performance metrics such as goals and assists. The architecture allows for future expansion into other data providers like *football-data.org* or *Opta* via plugin-based integration.

- **Internal REST API (FastAPI):** Communication between the React frontend and the Python core is managed through an asynchronous RESTful architecture. This JSON-based bridge handles requests such as GET /stats/{id} for player data and POST /team for squad validation.

5.3 Performance Considerations

To meet the high expectations of strategic simulation fans, several optimization strategies are employed:

- **Response Time:** By running validation logic in-memory within the Python core, the system targets a **response time of < 200ms** for team saving and validation. Player retrieval is optimized using indexed lookups in the SQLite database.
- **Scalability:** * **Modular Growth:** The use of **Factory** and **Strategy** patterns ensures that adding new rules or player types does not degrade performance with complex nested logic.
    - **Database Portability:** While currently using **SQLite**, the system uses SQLAlchemy ORM standards. This allows the platform to scale horizontally to a **PostgreSQL** environment within minutes if active users exceed 10,000+.
- **Optimization Strategies:**
    - **Singleton Pattern:** Critical objects like database managers and fetchers are maintained as single references in RAM to prevent memory leaks.
    - **O(1) Look-ups:** To prevent spam and duplicate processing, the API utilizes a set-based tekilleştirme (seen_pids) for efficient player verification.
    - **UI DOM Efficiency:** The frontend prevents render bottlenecks by limiting the number of player nodes rendered simultaneously (e.g., using slice for pagination)

6. Error Handling, Logging, and Testing

This section outlines how the system maintains stability through defensive programming, automated logging, and a design optimized for rigorous testing.

6.1 Exception Management (Graceful Degradation)

The system is designed to isolate and manage exceptions across different layers to ensure the application does not crash during runtime:

- **I/O and External API Failures**: If the external football-data.org or FPL API becomes unreachable during data fetching via fpl_fetcher.py, the script identifies the HTTP status code and logs a requests.HTTPError instead of crashing. In such

cases, the system enters a passive mode and continues to operate using the most recent cached Gameweek data from the SQLite database.

- **Data Integrity and Default Values**: To handle potential "NULL" values in database columns added later (e.g., threat or creativity), a defensive layer is implemented within the **Factory Pattern**. By using .setdefault() methods during object instantiation, missing keys are caught and assigned default values (e.g., 0.0), preventing TypeErrors or ValueErrors within the OOP models.
- **User-Input Errors**: Invalid team creation requests (e.g., exceeding budget) do not trigger system-level exceptions. Instead, the **Strategy Pattern** returns a detailed error reason as a string, which the FastAPI controller then transmits to the client as a controlled **HTTP 400 Bad Request** response.

6.2 Logging Mechanisms

The system utilizes the **Observer Pattern** to decouple core logic from logging requirements:

- The RealTimeUIUpdater observer class listens for events such as adding or removing players.
- Whenever a publisher (Subject/Team) broadcasts an event, the observer generates a time-stamped log in the system terminal (e.g., [UI] NEW EVENT - Type: PLAYER_ADDED).

6.3 Design for Testability

A primary objective of the OOP-based architecture is to facilitate comprehensive **Unit Testing**:

- **Loosely Coupled Layers**: Because validation strategies are decoupled from the FastAPI controller, developers can generate mock teams in the Python console to test thousands of scenarios in seconds.
- **End-to-End Testing**: Automated scripts like test_api_team.py are used to run load tests against active API endpoints.
- **Dependency Injection**: Validation rules are injected into the team object externally (e.g., my_team.add_validation_strategy(...)), allowing testers to intentionally set extreme values (e.g., Max_Budget = 0) to observe the system's breaking points.

7. Deployment and Installation Design

The system follows a **"Zero-Config"** deployment model, ensuring portability across any environment with Python 3.9+. It utilizes an **embedded SQLite** database, eliminating the need for external SQL server configurations.

- **Dependencies:** fastapi, uvicorn, requests.
- **Deployment Steps:**
    1. python database_setup.py (Initialize DB schema).
    2. python fpl_fetcher.py (Fetch and process live FPL data).
    3. python -m uvicorn api:app --port 8000 (Launch the web server).

8. Change Log and Future Work

8.1 Change Log

- **v1.x:** Established core OOP models and integrated the **Factory Pattern** for player instantiation.
- **v2.x:** Implemented the **Strategy Pattern** for rule validation and **Observer Pattern** for real-time event logging.
- **v3.x:** Developed the full-stack web architecture using **FastAPI** (backend) and **React** (frontend SPA).

8.2 Future Work

- **AI Predictions:** Integrating FDR and ICT data for predictive transfer recommendations.
- **Dynamic Formations:** Adding flexible tactical setups (e.g., 4-4-2, 3-4-3) via strategy enhancements.
- **Multi-User Ecosystem:** Implementing authentication and competitive mini-leagues with global leaderboards.
- **CI/CD Integration:** Establishing automated pipelines and migrating to cloud-based PostgreSQL for global scale